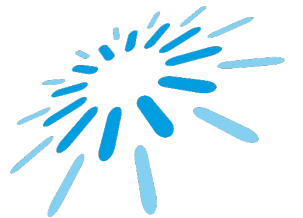




Collaboration, Quality, Test

Prof. Dr. Jóakim von Kistowski



TH Aschaffenburg
university of applied sciences

Recap:
What does **traceability** mean w.r.t. testing?

Chapter 2: Testing Throughout the Software Development Lifecycle



**Testing in the Context of a
Software Development Lifecycle**

Test Levels and Test Types

Maintenance Testing

Learning Objectives for Testing Throughout the Software Development Lifecycle

(according to Certified Tester Foundation Level Syllabus, English Version V4.0)

- **2.1 Testing in the Context of a Software Development Lifecycle**
- FL-2.1.1 (K2) Explain the impact of the chosen software development lifecycle on testing
- FL-2.1.2 (K1) Recall good testing practices that apply to all software development lifecycles
- FL-2.1.3 (K1) Recall the examples of test-first approaches to development
- FL-2.1.4 (K2) Summarize how DevOps might have an impact on testing
- FL-2.1.5 (K2) Explain the shift-left approach
- FL-2.1.6 (K2) Explain how retrospectives can be used as a mechanism for process improvement

Learning Objectives for Testing Throughout the Software Development Lifecycle

(according to Certified Tester Foundation Level Syllabus, English Version V4.0)

- **2.2 Test Levels and Test Types**
- FL-2.2.1 (K2) Distinguish the different test levels
- FL-2.2.2 (K2) Distinguish the different test types
- FL-2.2.3 (K2) Distinguish confirmation testing from regression testing
- **2.3 Maintenance Testing**
- FL-2.3.1 (K2) Summarize maintenance testing and its triggers

Software Development and Software Testing

- Software development life cycle model (→ or process model)
 - describes activities for each phase of a software development project
 - describes the logical and temporal relationship of the activities
- Different software development lifecycle models
 - each requires different approaches to testing
- Two main categories:
 - **Sequential** development models
 - **Iterative and incremental** development models

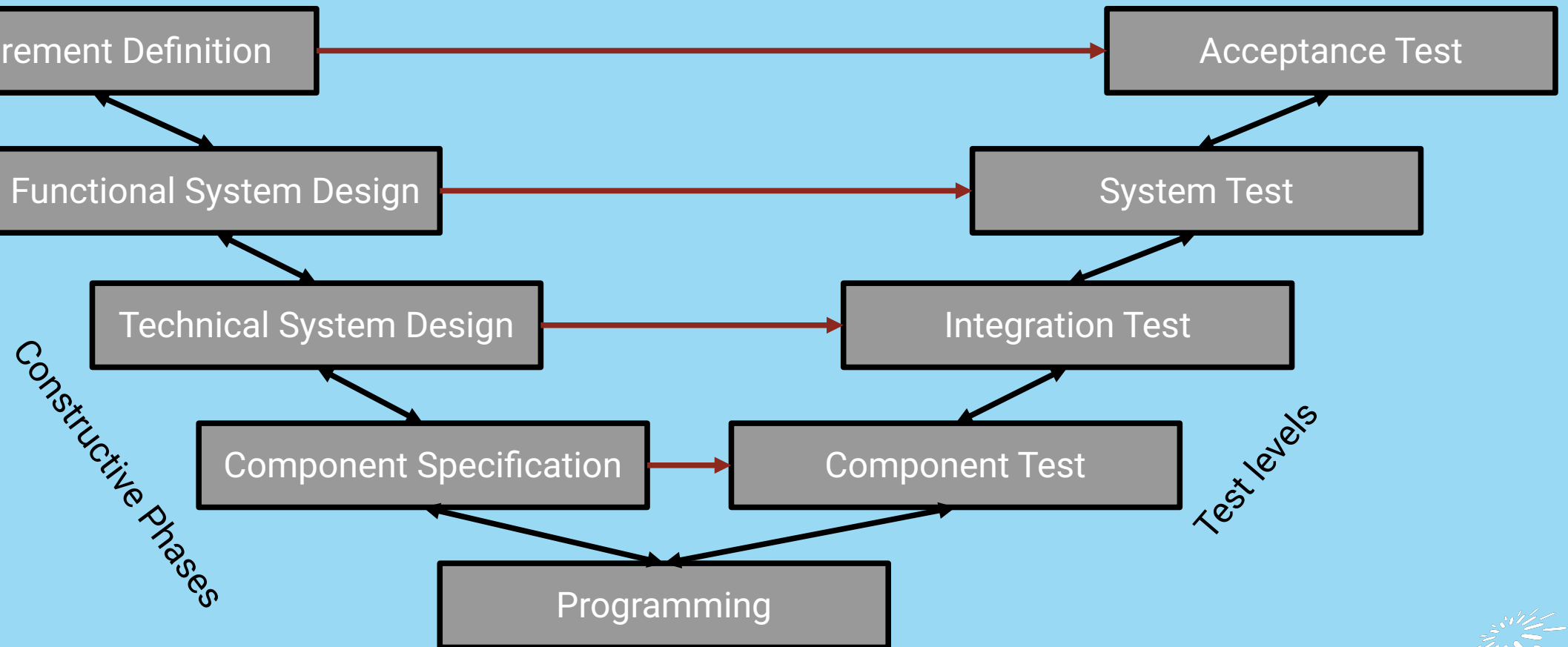
Question

**Which process models do you remember from
Software Engineering?**

Sequential Process Models

V-Model

V-Model (Sequential Development Model)



Task: Discuss in Groups

Find the V-Model's main properties.

- What are the constructive and analytic activities?
- When do verification and validation take place?

V-Modell – Constructive Phases (1/2)

Constructive Phases

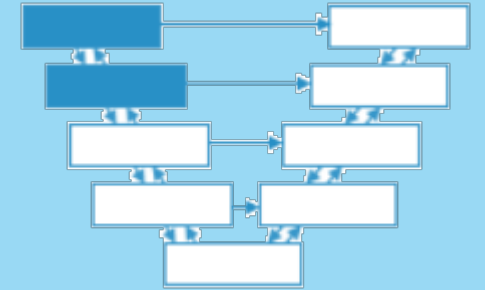
- Constructive activities in the left branch
- Software system to be described in ever increasing detail

Requirements Definition

- Collecting requirements from client or system user
- Defining purpose and non-functional characteristics of software system

Functional System Design (or just “System Design”)

- Mapping requirements to system functions and dialog processes



V-Modell – Constructive Phases (2/2)

Technical System Design (or “Architecture Design”)

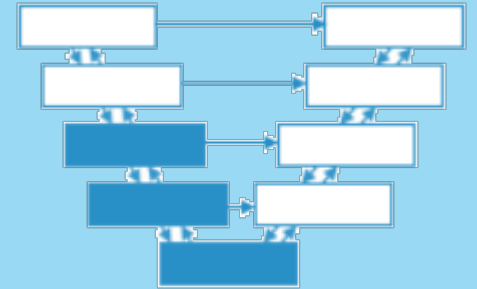
- Designing the system’s technical implementation
- Defining the system <-> environment interfaces

Component Specification

- Defining subsystem tasks, behavior, internal structure, and interfaces

Programming

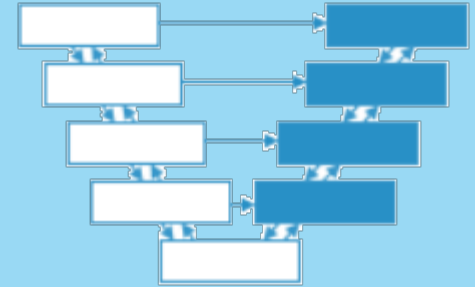
- Implementation of each component (module, unit, class, etc.)



V-Modell – Test levels (1/3)

Motivation

- Finding errors is easiest on the same abstraction level
- Right side branch has a corresponding test level for each constructive phase



V-Modell – Test levels (2/3)

Who's who?

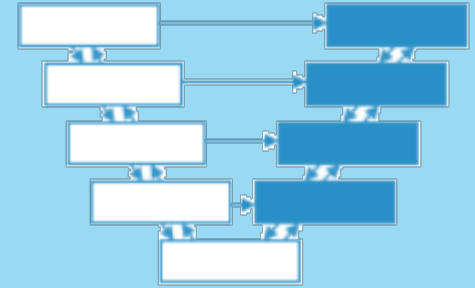
**Acceptance
Test**

**System
Test**

**Integration
Test**

**Component
Test**

- "I check whether the system as a whole fulfills the specified requirements."
- "I am there to check whether each software or hardware component fulfills its specification."
- "I check whether the components interact as defined in the technical system design."
- "I check whether the system has the agreed performance characteristics from the customer's point of view."

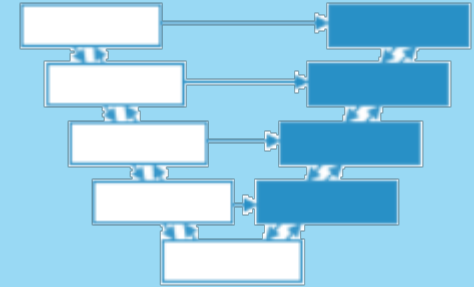


V-Modell – Test levels (3/3)

- **Component test:** Does every software or hardware component fulfill its specification?
- **(Component) Integration test*:** Do components interact as specified in the technical system design?
- **System test:** Does the system as a whole fulfill the specified requirements?
- **Acceptance test:** Does the system exhibit agreed performance characteristics from the customer's point of view?

Additional type of integration test:

- **System integration test:** after system test, tests the interaction of different systems or between hardware and software
- **Unit Integration Test** = Component Integration Test



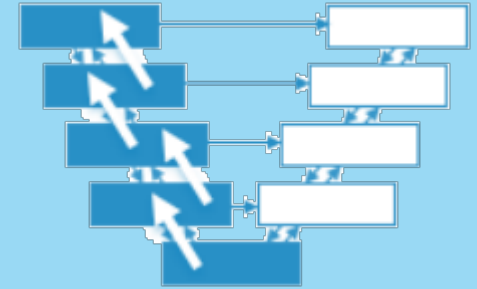
V-Modell – Verification

Verification

- Related to specific development phase
- Proof of correctness and completeness with regard to direct specification

Goal of verification

- **Specifications implemented correctly?**
- Independent of the intended purpose or use of the product!



V-Modell – Validation

Validation per test level:

- Do development results meet requirements at the given level?

Questions for validation:

- Does a (partial) product solve a defined task?
- Is it suitable for its intended use?

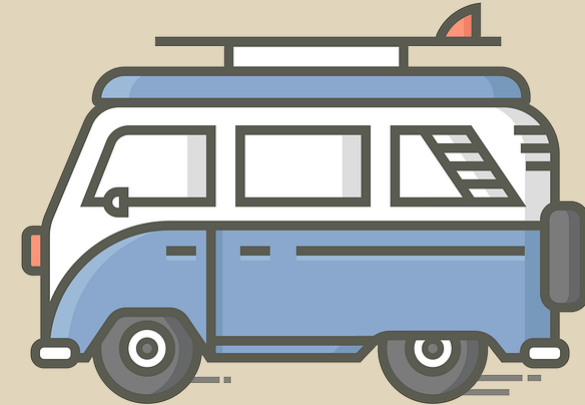


Question

When does testing start when using the V-Model?

Question

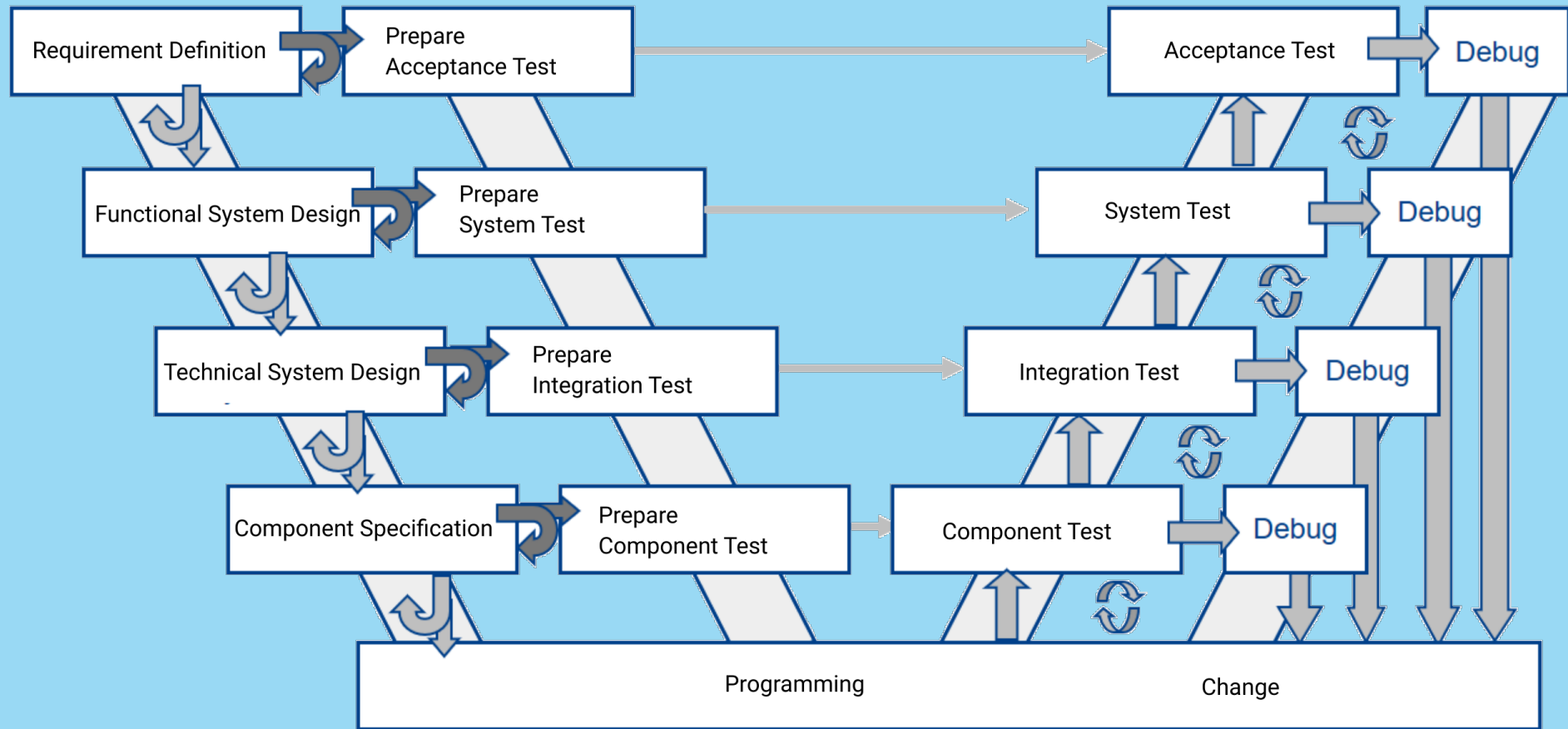
Vacation time:



When do you pack?

What determines when you start packing?

V-Modell – Further Development



after Spillner/Roßner/Winter/Linz
Praxiswissen Softwaretest - Testmanagement
dpunkt, 2014, Chapter 3.3.2



Review, Previews,
Documents



Test cases,
Test environments



test, debug,
change, re-test

Iterative-Incremental Process Models

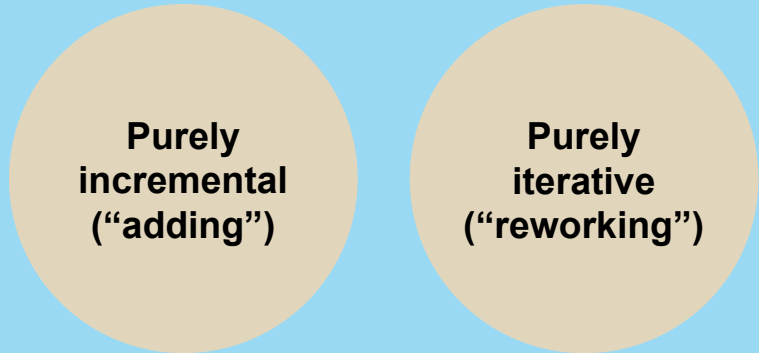
Iterative-Incremental Development Models (1/2)

- Functions/features specified, designed, implemented and tested in cycles
- System not created “in one piece”, but in increments (version statuses, interim deliveries)

Per iteration:

- new features or functions
- Changes to existing functions
- Improvement of system quality

Iterative-Incremental Development Models (2/2)



**Purely
incremental
("adding")**

**Purely
iterative
("reworking")**

- System (incl. goals) is created gradually using increments
- Assumption: Features or project scope can be changed at any time
- Features specified, designed, implemented and tested in cycles (with a fixed duration)
- Per iteration: executable, potentially deliverable software
- Goals of the overall system clear from the start
- Assumption: partial results no longer change
- System is cyclically revised, detailed and supplemented

Task: Discuss in Groups

How does testing in iterative process models differ from testing in sequential process models?

Testing in Iterative-Incremental Development Models (1/2)

- Tiny W-Model per iteration
- Test phases and test stages often overlap

Tips:

- Test each feature on several test levels if possible
- Use reusable tests per increment and iteration
- New functionality: additional tests
- Per increment in an iteration: run through different test levels
- Continuous: perform integration tests and regression tests
- Verification and validation for each increment

Testing in Iterative-Incremental Development Models (2/2)

- In small intervals ...
 - integrate software: continuous integration
 - deploy software: continuous deployment
- Important: **Test automation** on several test levels!

Tips for high quality Tests

- Test activity for each development activity
- Start test activities as early as possible
 - Start test analysis and design in parallel with the development stage
- Involve testers at an early stage:
 - Define the requirements
 - Software design
 - Review process of requirements, design documents, user stories, etc.
- Software development models
 - Not applicable “out of the box
 - Adapt to project and product characteristics (number of test stages, number and length of iterations, etc.)

Chapter 2: Testing Throughout the Software Development Lifecycle

Component Test



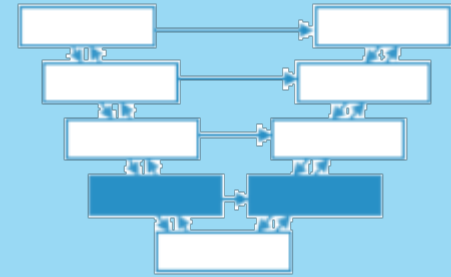
**Testing in the Context of a
Software Development Lifecycle**

Test Levels and Test Types

Maintenance Testing

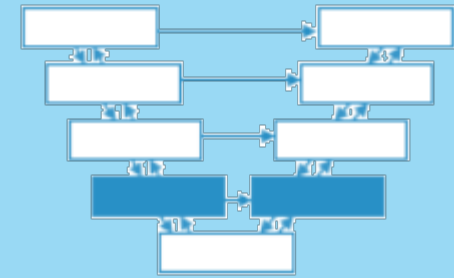
Component Test – Terms

- Component test (first test stage): Software modules tested for the first time
- Component: Name for the smallest software units depending on the programming language
 - Modules, units or classes (object-oriented programming)
 - corresponding tests: module, unit or class test
- Abstract from programming language: Component or software module
 - Test of a software module: component test



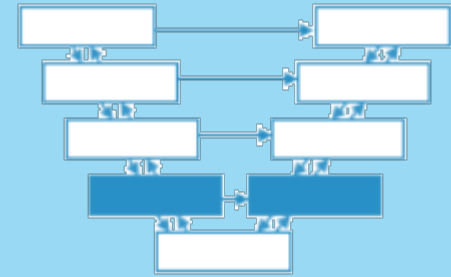
Component Test – Test Objectives (1/4)

- Task of component tests:
 - Does the test object implement the required functionality correctly and completely?
 - Functionality synonymous with input/output behavior of test object
- Further test objectives:
 - Finding defects in the component
 - Prevent defects from being passed on to higher test levels
 - Risk reduction
 - Verification of the (non-)functional behavior of the component
 - Creating confidence in the quality of the component



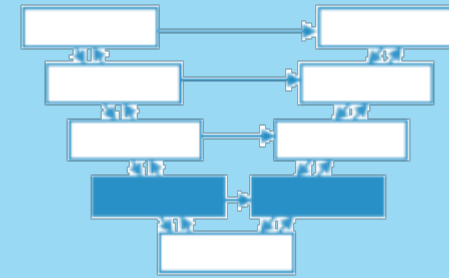
Component Test – Test Objectives (2/4)

- Check all component properties (including non-functional ones),
 - ...which influence the quality of the component
 - ...which can no longer be (easily) tested at higher test levels
 - Examples: efficiency, maintainability



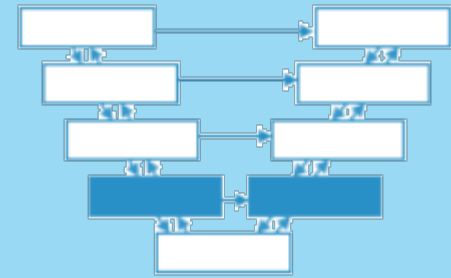
Component Test – Test Objectives (3/4)

- Example test NFR: **Robustness**
 - Each software component interacts with neighboring components
 - Components can be addressed or used incorrectly
 - → do not immediately stop service and crash the system
 - Instead: Intercept error situation and react “reasonably” / robustly
- Example test NFR: **Efficiency**
 - How economically does the component use available resources?
 - Precisely measure sub-criteria (e.g. memory consumption, response time) in the test



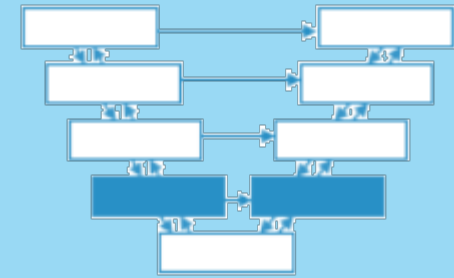
Component Test – Test Objectives (4/4)

- Example test NFR: Test for maintainability
 - How easy or difficult is it to change / further develop the program?
 - How much effort does it take to understand the program and context?
 - Test aspects: Code structure, modularity, commenting of the code, comprehensibility and up-to-dateness of documentation, etc.
- Maintainability cannot be checked by dynamic tests
 - Instead: Analysis of program text and specification, static test and (code) review
 - If possible, carry out analyses as part of the component test



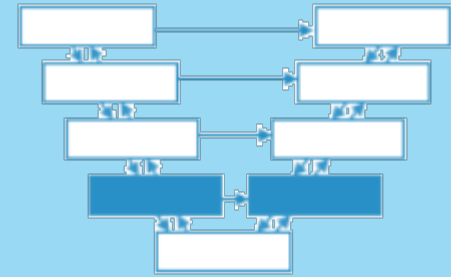
Component Test – Test Basis

- Work results of detailed design, program code
- All documents that exist for the component
 - Data models, class models, behavior models (e.g. sequence diagrams)
 - Pre- and post-conditions of the operations, invariants of the component
- Agile projects are often test-driven (test-first, test-driven)
 - Automated test cases are specification and executable test at the same time



Component Test – Test Objects

- **Test objects: System software components**
 - Components, units or modules
 - Code, data structures, classes
 - Database modules
- **Each software module tested in isolation from others**
 - No external influences
 - Failure detected? Cause in tested component!
- **Component to be tested composed of several modules?**
 - Important: check component-internal aspects
 - Do not check the interaction with neighboring components!



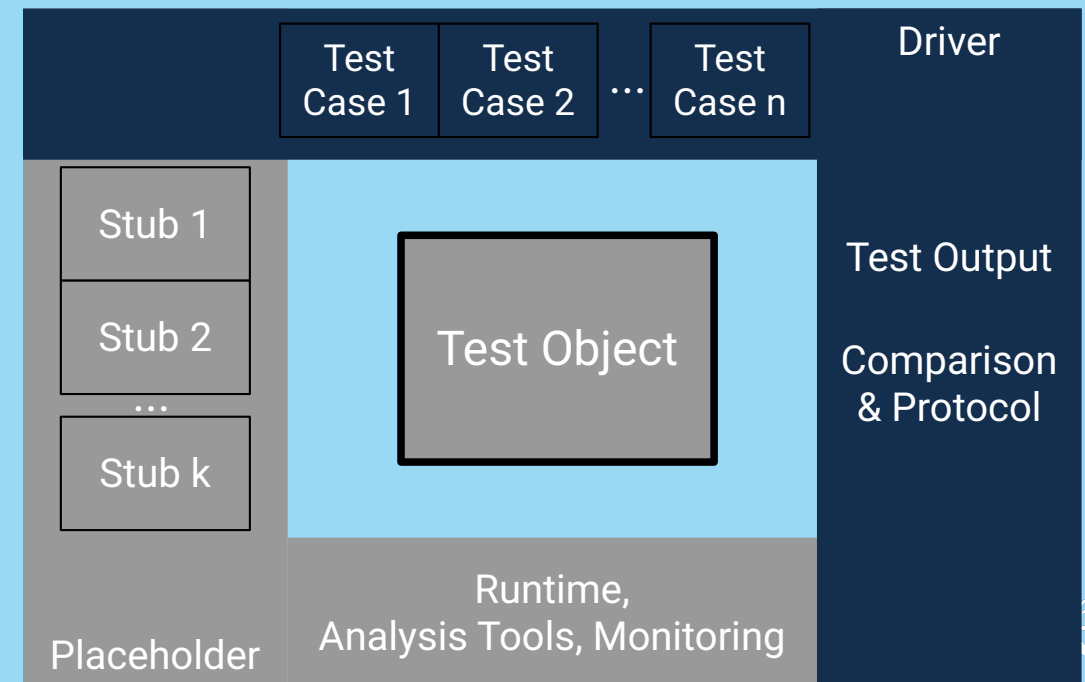
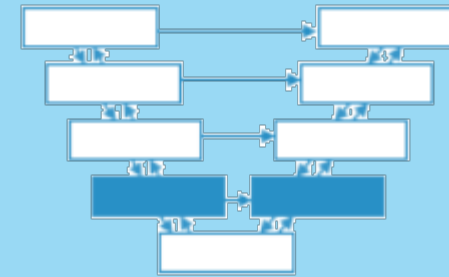
Component Test – Test Environment (1/6)

Test environment consists of

- **(Test) Driver**
Calls the test object

and/or

- **Stub (Mock/Dummy)**
Simulation of services imported by test object

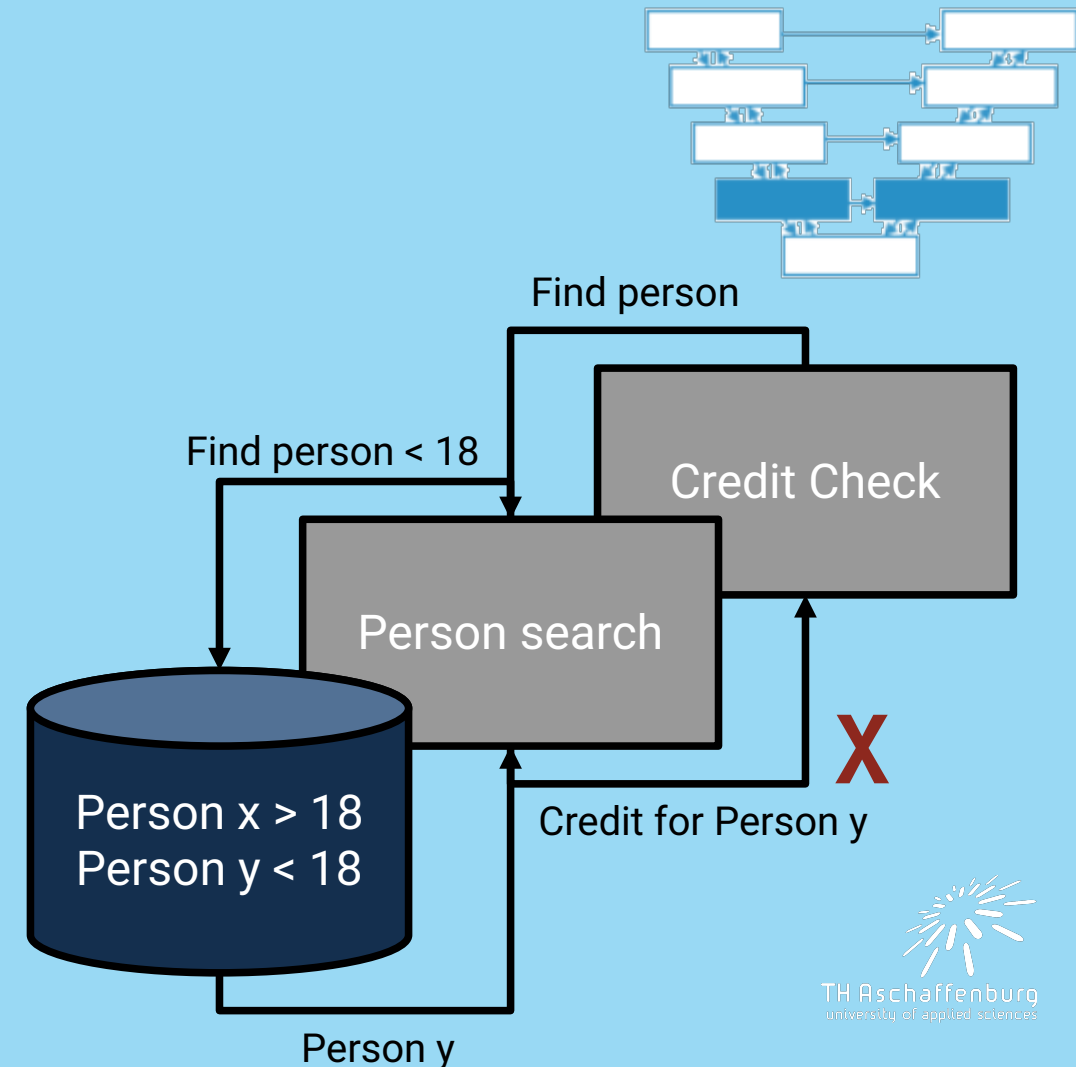


Component Test – Test Environment (2/6)

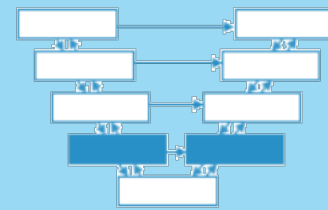
Test cases (simplified)

- Test case 1:
 - **Positive:** Grant credit to person with age ≥ 18
Calls the test object
- Test case 2:
 - **Negative:** Decline Credit to person with age < 18

What could be a problem with this approach?



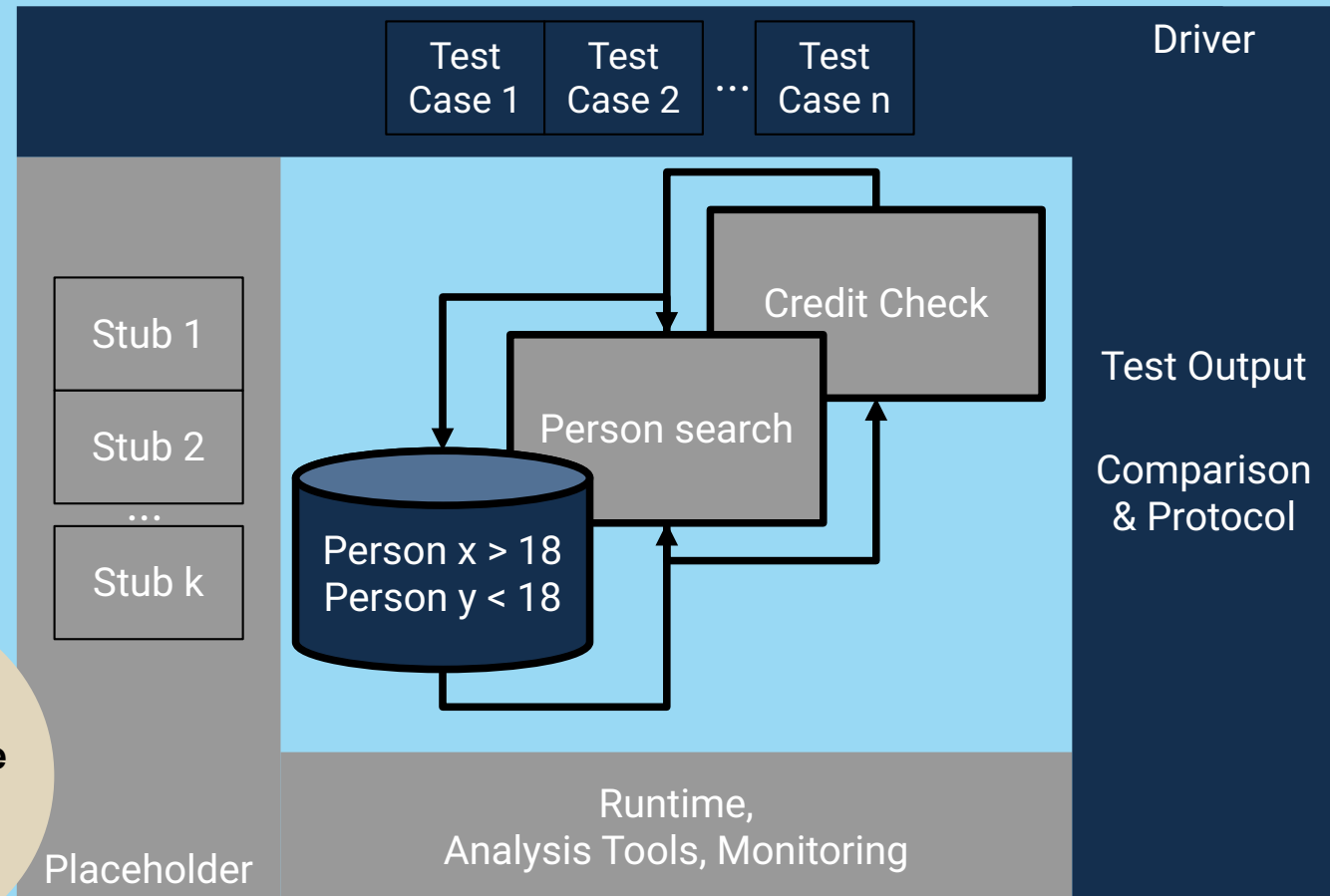
Component Test – Test Environment (3/6)



Test environment consists of

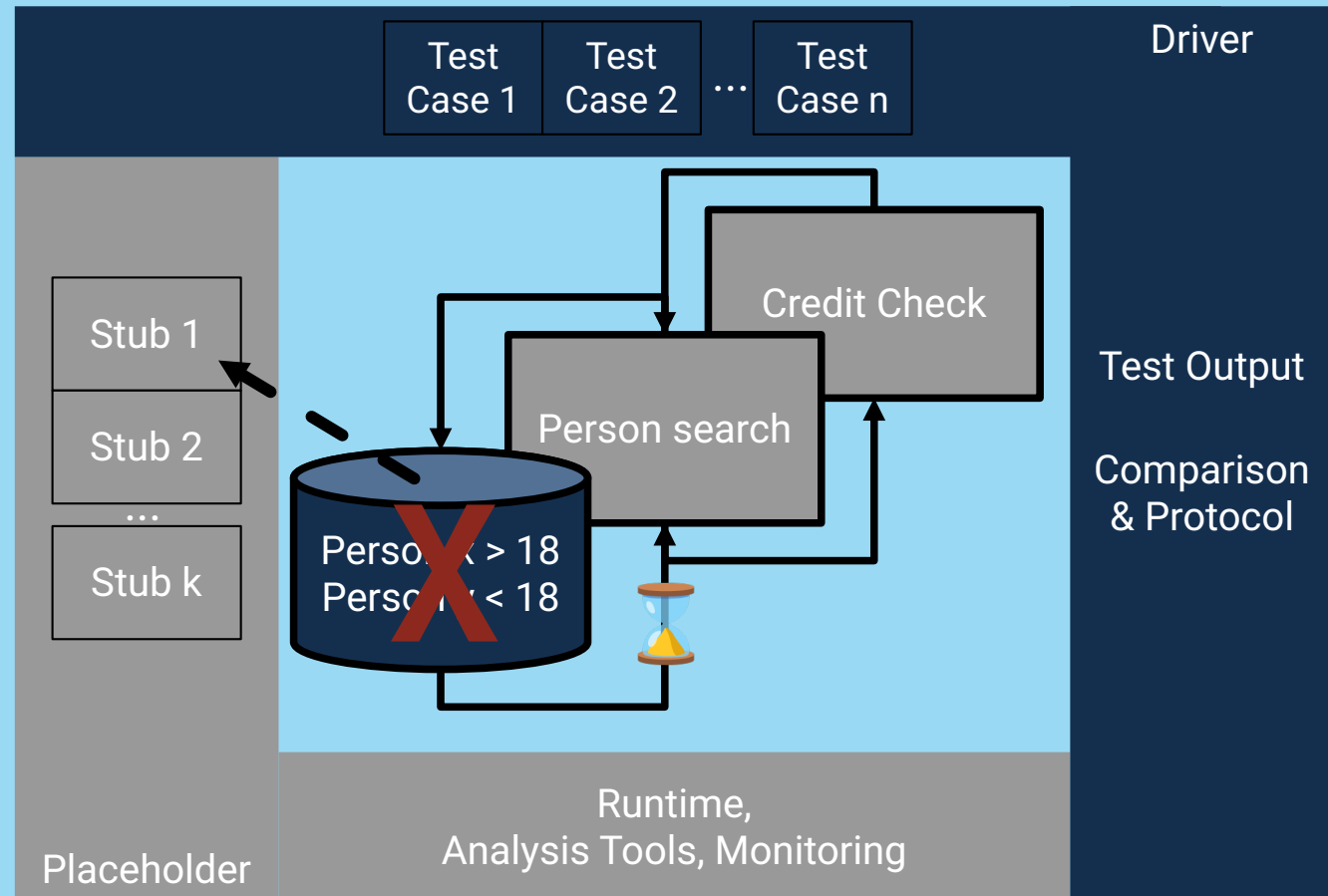
- **(Test) Driver**
Calls the test object
- **Positive:**
Grant credit to person with age ≥ 18
- **Negative:**
Decline Credit to person with age < 18

What could be simulated?

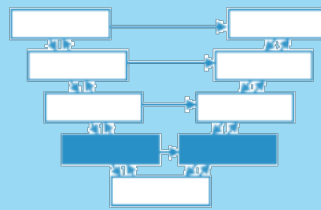


- (Test) Driver

- **Positive:**
Grant credit to person with age ≥ 18
- **Negative:**
Decline Credit to person with age < 18
- **Stub (Mock/Dummy)**
Simulation of services imported by test object

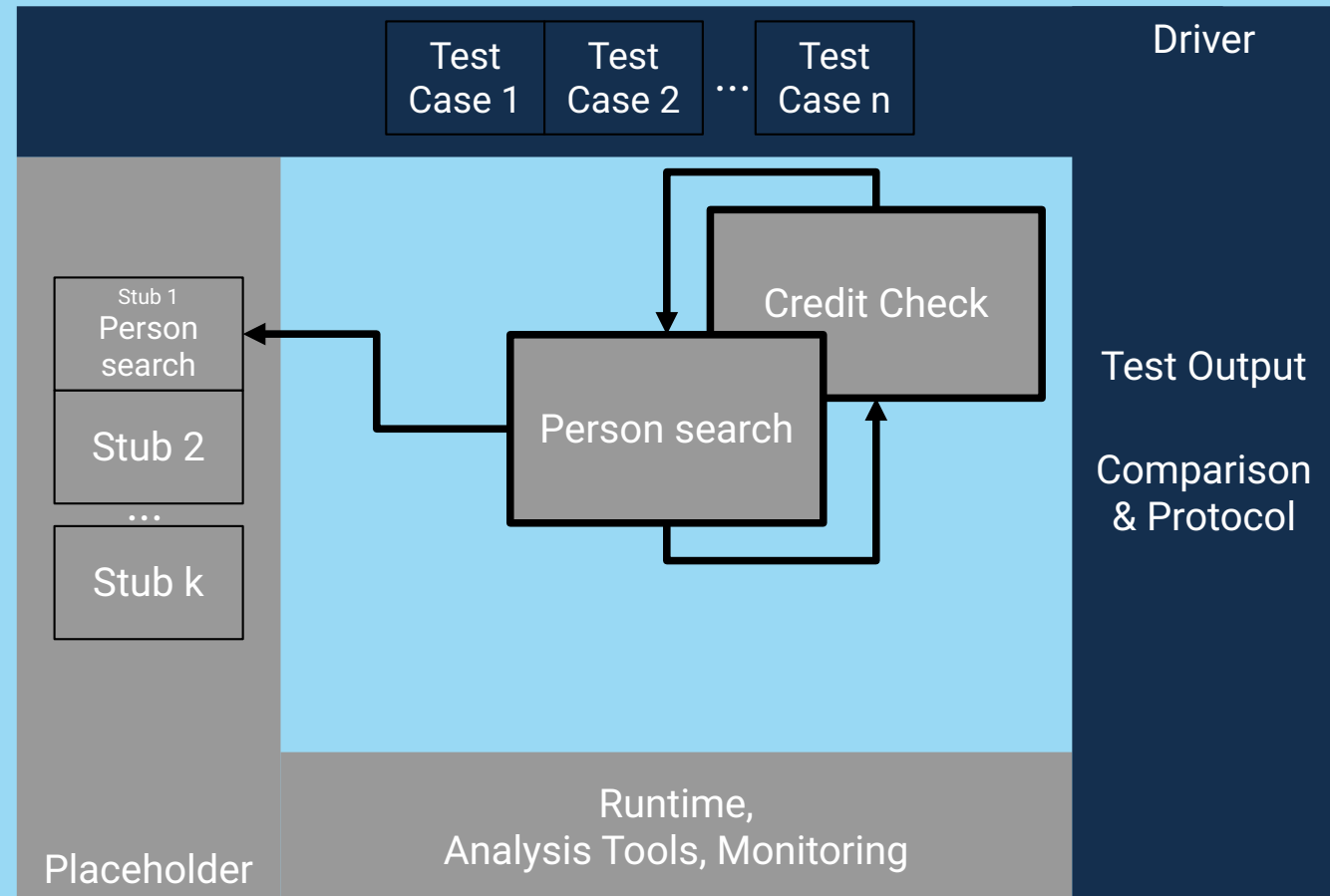


Component Test – Test Environment (5/6)



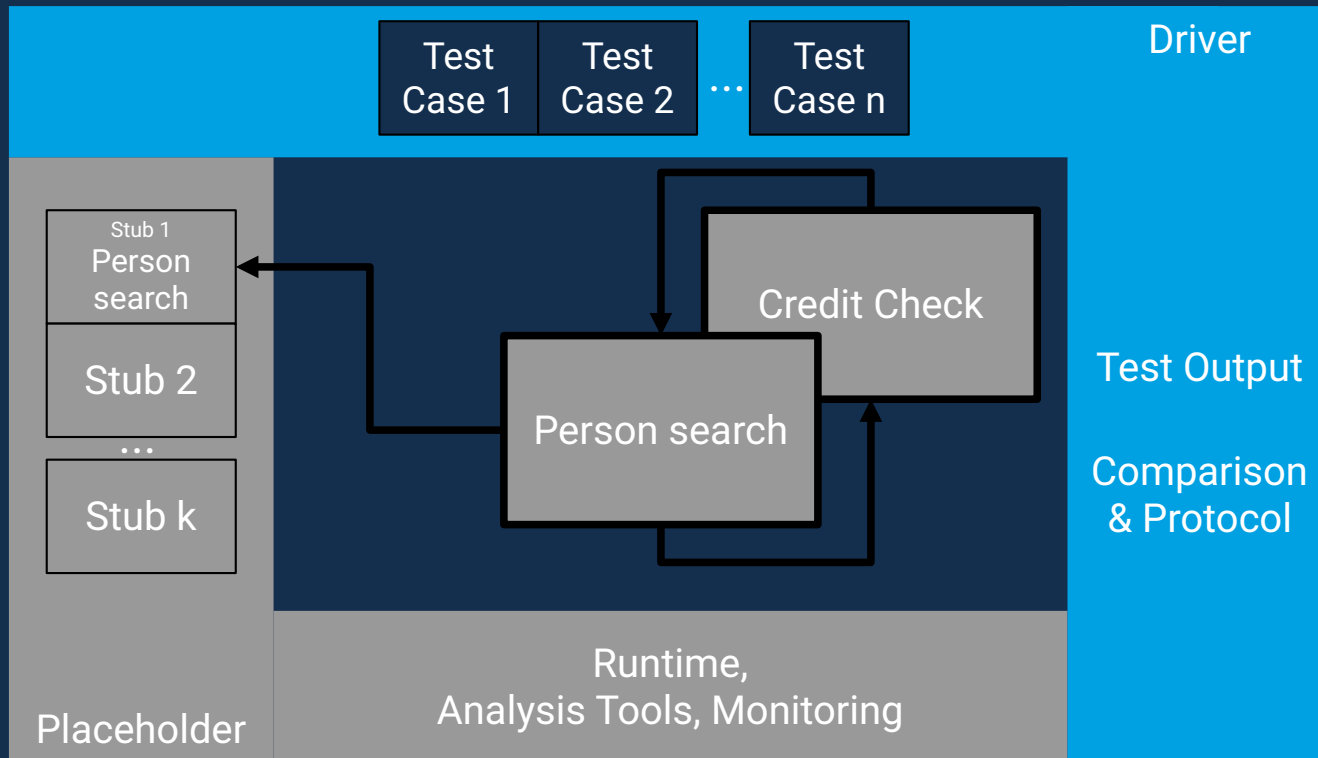
Test environment consists of

- **(Test) Driver**
Calls the test object
- **Positive:**
Grant credit to person with age ≥ 18
- **Negative:**
Decline Credit to person with age < 18
- **Stub (Mock/Dummy)**
Simulation of services imported by test object
- Simulate person search;
prevent actual search,
data base queries, ...



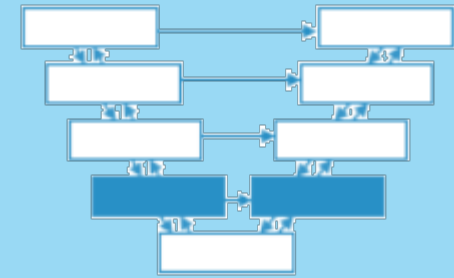
Task: Discuss in Groups

What are the advantages of such an approach?



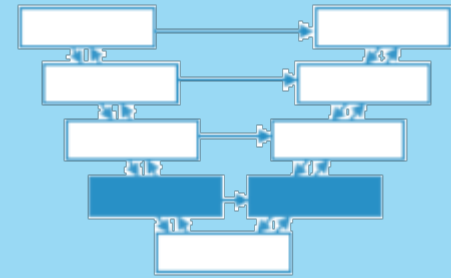
Component Test – Test Environment (6/6)

- This test level: development-related work
- Developer know-how required for test environment
- Code of test object / interface must be available
 - Enables calling of test object
- Component tests often carried out by developers: “Developer test”
- Developers
 - develop code for component
 - write tests
 - execute tests
- Agile development: writing automated component test cases also possible **before** writing application code (test-driven development, TDD)



Component Test – Test Strategy (1/2)

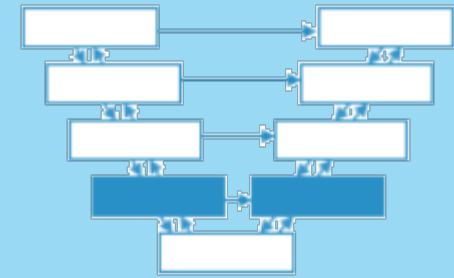
- Access to the program code → Component test = white-box test
- Design test cases using knowledge of internal aspects
- Presence of program code during test execution
- Observe program variables during execution
- Result: Correct or incorrect behavior of the component can be easily identified



Component Test – Test Strategy (2/2)

In practice: Component testing as a combination of black-box and white-box test

- Derive test cases from component requirements
- Use coverage of structural elements as end criterion
- Real software systems often have thousands of components
 - Access to code may be limited to selected components
- During integration, components are merged into larger units
Often during component testing, only composite components visible
- These test objects are too large for analysis at code level
- Test with focus on elementary / composite components?
- Define in integration planning and test planning



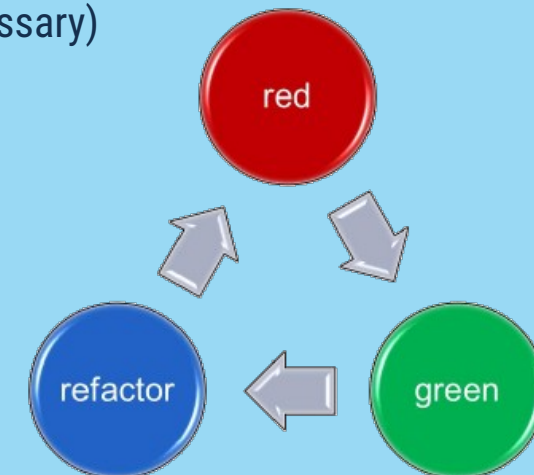
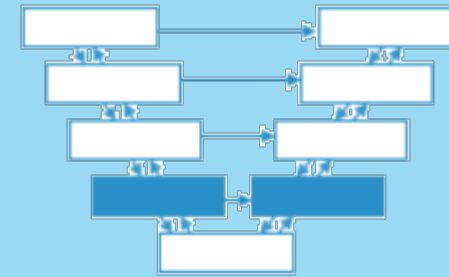
Component Test – Test First Approach (more on that in the future)

Agile projects: test-driven work

- Design & automate test cases first, then program components
- Iterative approach: improve code until tests show no failures
- Test-driven development (TDD):
A software development technique in which the test cases are developed, automated and then the software is developed incrementally to pass those test cases.

(ISTQB Glossary)

- Notes:
 - Test cases: Component specification (test basis) and “executable” test specification
 - First write test (make it red), then write code (make it green), then refactor code (make it blue)



What are advantages/disadvantages of TDD?

Quiz

Typical test objects in a component test:

a) of-the-shelf components

b) operations manuals

c) classes

d) overarching business processes

Typical test objects in a component test:

a) of-the-shelf components
(→ integration test)

b) operations manuals
(→ system test)

c) classes
(→ component test)

d) overarching business
processes
(→ acceptance test)

Goal of a component test:

a) find defects in interfaces and in interaction between components

b) gain client's/user's trust in the system

c) classes find defects in software parts that can be tested separately

d) gain client's/user's trust in certain non-functional properties

Goal of a component test:

a) find defects in interfaces and in interaction between components
(→ integration test)

b) gain client's/user's trust in the system
(→ acceptance test)

c) classes find defects in software parts that can be tested separately
(→ component test)

d) gain client's/user's trust in certain non-functional properties
(→ acceptance test)

Question

Stubs are:

a) placeholders simulating parts of the system

b) test data required during component testing

c) control the test object during component testing

d) simplify acceptance testing by simulating parts of the system

Question

Stubs are:

a) placeholders simulating parts of the system

b) test data required during component testing

c) control the test object during component testing

d) simplify acceptance testing by simulating parts of the system

Typical failures that can be found by a component test:

a) system incomplete

b) gain system does not behave as specified in user manual

c) unhandled failures in comm. between components

d) error in module logic

Typical failures that can be found by a component test:

a) system incomplete
(→ acceptance/system test)

b) gain system does not behave as
specified in user manual
(→ acceptance/system test)

c) unhandled failures in comm.
between components
(→ integration test)

d) error in module logic
(→ component test)

Who (primarily) runs component tests?

a) developers

b) test managers

c) users

d) UX experts

Question

Who (primarily) runs component tests?

a) developers

b) test managers

c) users

d) UX experts

Take Away – Chapter 2 – Component Test (1/2)

- Mindset - The earlier a defect is found, the ...
 - cheaper its localization
 - and its elimination
- Component tests can relate to both functional and non-functional properties of a component
- Reminder, NFRs: Functionality, reliability, usability, IT security, compatibility, portability, maintainability and performance efficiency [according to ISO 25010]



Take Away – Chapter 2 – Component Test (2/2)

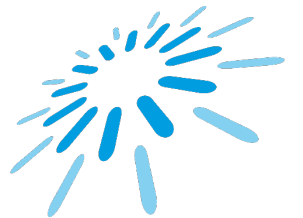
- Component tests are primarily carried out by developers, development expertise is required.
 - But: Test know-how is also required
- Stubs and drivers facilitate (automated) component testing.
- Test-first and test-driven development are state of the art methods, especially in iterative/incremental process models.





Until next time!

Collaboration, Quality, Test
Prof. Dr. Jóakim von Kistowski



TH Aschaffenburg
university of applied sciences